프론트엔드 관련/프로젝트

[React] 우리집 고양이도 할 수 있는 카카오 로그인 구현(with firebase OAuth 2.0 소셜 로그인)

by ash9river 2024. 8. 12.

React Firebase Kakao Authentification with OAuth

지난 글에서 구글 소셜 로그인을 해보았는데, 이어서 이번에는 카카오 소셜 로그인을 구현해보자.

[React] 구글 ...

React Firebase Google...

ash9river.tistory.com

분류 전체보기 (128)

컴퓨터 구조 (14)

React 완벽 가이드 (21)

프론트엔드 관련 (25)

기초 (4)

프로젝트 (13)

이슈 (5)

기타 (3)

알고리즘 (59)

백준 (50)

기타 (6)

공지사항

최근글 인기글

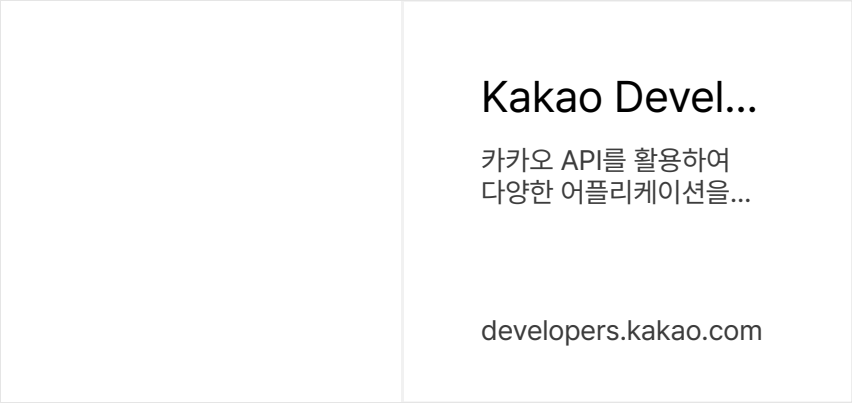
구글 소셜 로그인과는 다르게, 리다이렉션을 사용해서 만들 건데, REST API 방식을 활용한다. REST API 방식을 활용하기 때문에 스크립트 태그를 추가할 필요가 없다.

하지만 javascript sdk1의 팝업 방식을 손쉽게 구현하기에는 무리가 있다.

심지어 sdk2로 넘어가면서, sdk2에서는 로그인 팝업 방식을 지원하지 않는데, 만약 팝업 방식으로 로그인 기능을 만들고 싶으면 따로 찾아보자.

1. 카카오톡 애플리케이션 등록

카카오톡 개발자 사이트에 들어가서 상단의 내 애플리케이션에 들어가자.



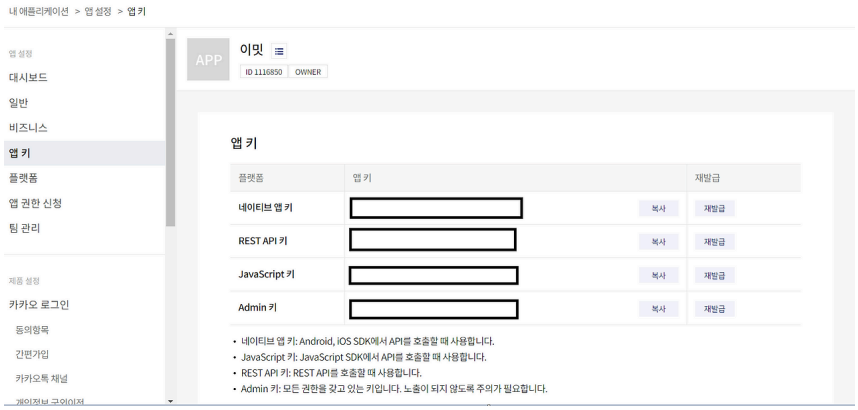
애플리케이션 추가하기 버튼을 눌러서 적당히 입력하자.

그리고 3 가지를 해야한다.

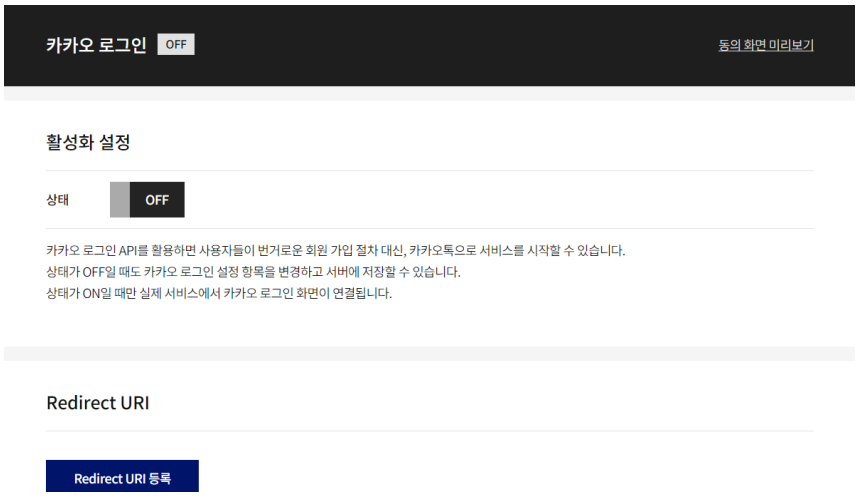
맥도날드 더블 쿼터파...
2025.10.27
맥도날드 신메뉴 페퍼...
2025.09.18
광고 댓글이 너무 많이 달리는데
2025.08.08
백준 16954 움직이는 ...
2025.08.08
백준 23559 밥 C++
2025.08.08
최근댓글
map, filter 좀 풀어줘봐 ...
vs 피자루트
들렸다 갑니다~
태그
오블완, 푸하하,
티스토리챌린지
전체 방문자
13,419
Today : 12
Yesterday : 17

1. 앱설정의 앱 키 기억
2. 카카오 로그인 설정 활성화 및 Redirect URI 등록
3. Client Secret 활성화

먼저, 1번을 위해 프로젝트를 클릭해서 앱 설정에 들어가자.

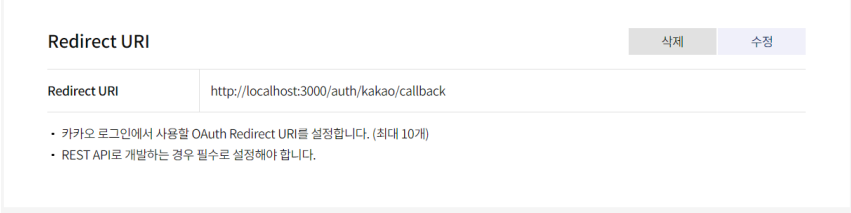


다음과 같이 앱 키가 나와있는데, 다른 키는 사용하지 않고, REST API 키를 활용할 것이다. 잘 기억해두자.

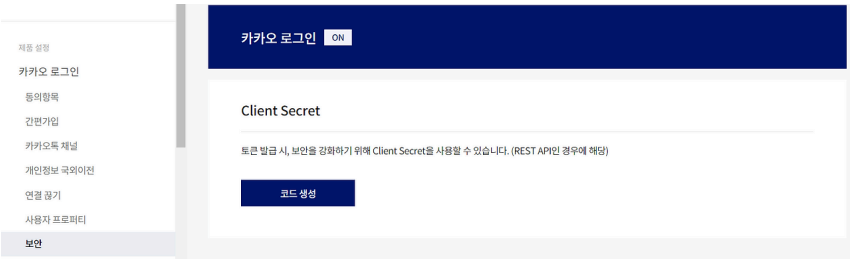


그 다음 두번째로, 제품 설정에서 카카오 로그인을 활성화시키고, 같이 나오는 OpenID Connect 또한 활성화시킨다.

그리고 Redirect URI을 등록한다.



보통 http://localhost:3000/auth/kakao/callback로 많이 설정하여서, 나도 똑같이 URI를 설정하였다.



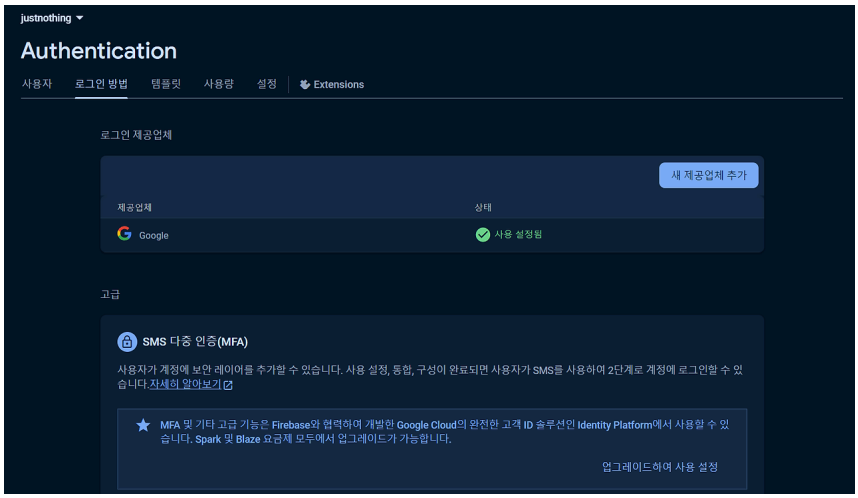
마지막으로, 제품설정 > 카카오 로그인 > 보안에서 Client Secret을 활성화시킨다.(활성화 상태도 사용함으로 만들어야 한다.)

참고로, 만약에 Kakao SDK for JavaScript, 카카오톡 공유, 메시지 API를 사용한다면, 앱설정 > 플랫폼 > Web에서 Web 플랫

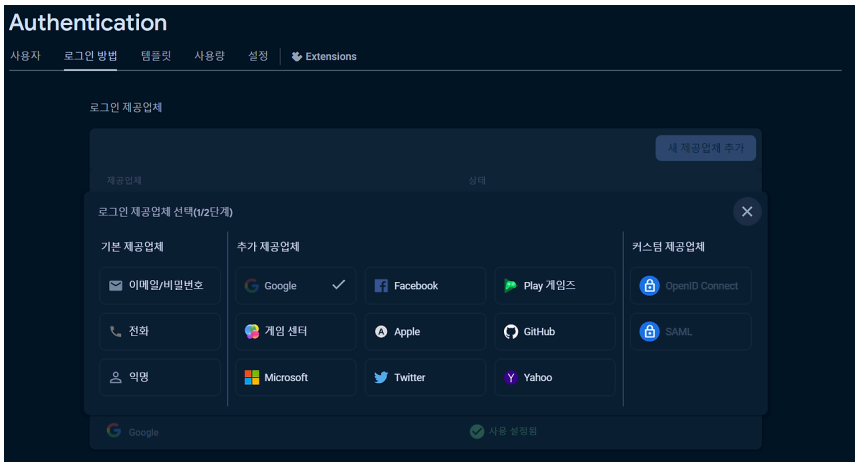
폼을 등록해야 한다.(이 글에서는 SDK를 이용하지 않고 REST API를 사용한다.)

이제 파이어베이스 콘솔을 열어보자.

2. 파이어베이스 설정



파이어베이스의 Authentification에 들어가서 새 제공업체를 추가한다.



이때, 새 제공업체는 OpenID Connect를 선택하여야 하는데, 파이어베이스에서 OpenID Connect를 사용하면, 요금제를 업그레이드해야되는 문제가 있지만, 그래도 개인 프로젝트에서는 별도의 추가 과금 없이 사용할 수 있다.

무료(Spark)

무료 (Spark) 요금제의 프로젝트에서는 대부분의 로그인 제공업체에 대해 일일 활성 사용자(DAU)가 3,000명으로 새로 제한됩니다. 일일 활성 사용량은 24시간 동안의 순 사용자 로그인 수를 기준으로 계산됩니다.

제공업체	새 한도	이전 한도
이메일, 소셜 미디어, 익명, 커스텀	DAU 3,000명	무제한
SAML, OpenID Connect	DAU 2명	해당 사항 없음

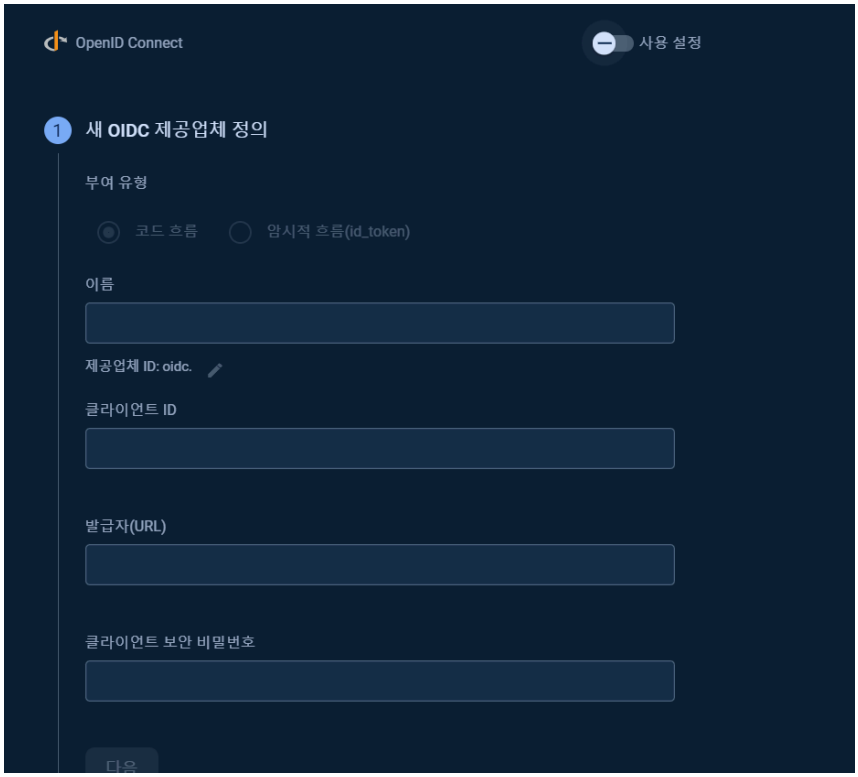
사용한 만큼만 지불(Blaze)

Blaze 요금제의 프로젝트 가격은 월간 활성 사용자 수(MAU)를 기준으로 하며 무료 등급 사용자 50,000명을 포함합니다. 활성 사용자는 해당 결제 기간 내에 계정을 사용한 사용자입니다.

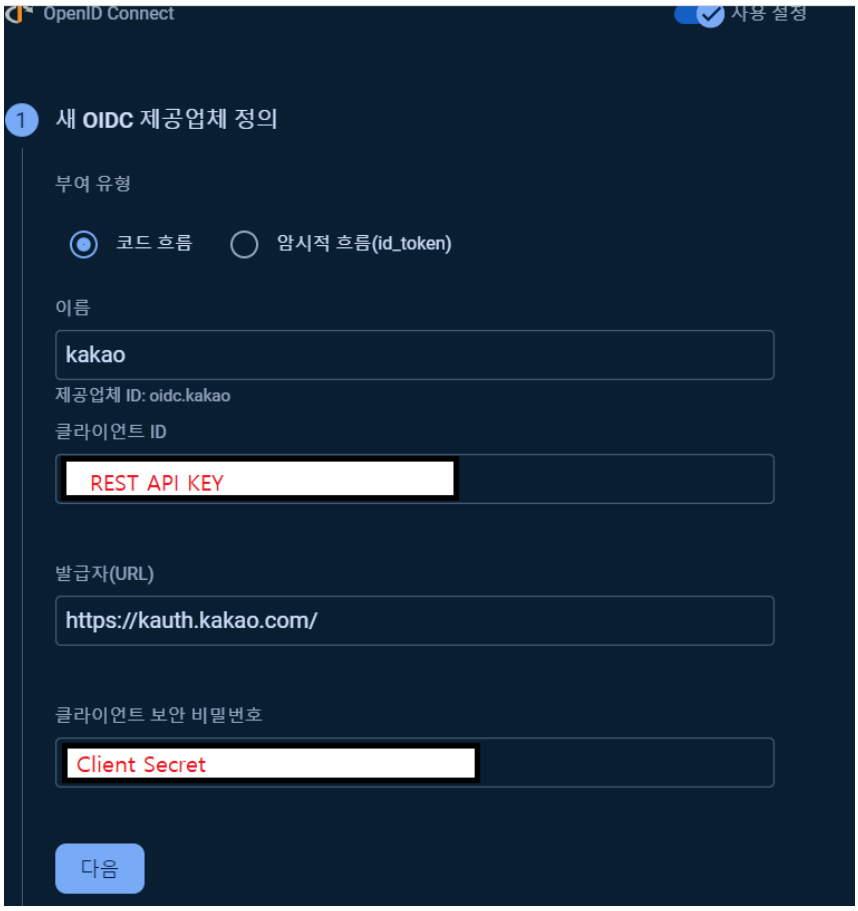
제공업체	무료 등급	무료 등급 초과 시 MAU당 비용($)
이메일, 소셜 미디어, 익명, 커스텀	MAU 0~49,999명	MAU당 0.0025~0.0055
SAML, OpenID Connect	MAU 0~49명	MAU당 0.015

과금 기준

요금제를 활성화시키면 다음과 같은 입력창이 뜬다. 사용설정을 활성화하자.



여기서 제공업체의 이름은 아무렇게나 해도 되나, 그냥 kakao로 하자.

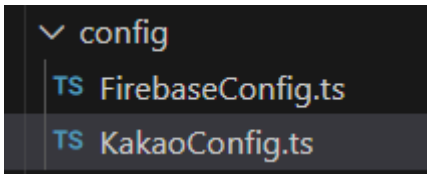


클라이언트 ID는 REST API 키를 사용하고, 클라이언트 보안 비밀번호는 Client Secret을 입력하자.

그리고 발급자는 다음과 같은 URL를 사용한다.

```
https://kauth.kakao.com/
```

그리고 따로 config 파일과 env.를 이용해 다음과 같이 config를 설정한다.

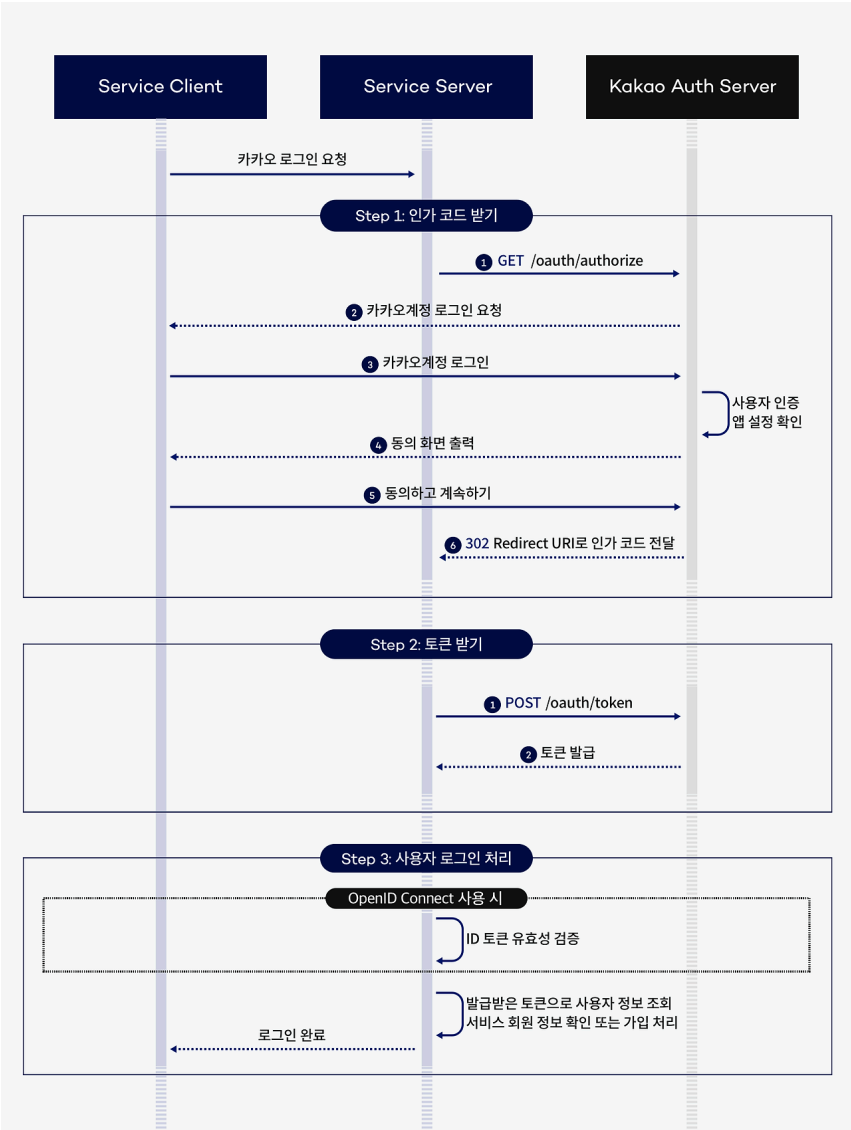



```
export const REST_API_KEY =
process.env.REACT_APP_REST_API_KEY as string;
export const REDIRECT_URI =
process.env.REACT_APP_REDIRECT_URI as string;
```

```
export const KAKAO_AUTH_URL =
`https://kauth.kakao.com/oauth/authorize?
client_id=${REST_API_KEY}&redirect_uri=${REDIRECT
_URI}&response_type=code`;
export const CLIENT_SECRET =
process.env.REACT_APP_CLIENT_SECRET as string;
export const KAKAO_APP_KEY =
process.env.REACT_APP_KAKAO_APP_KEY as string;
```

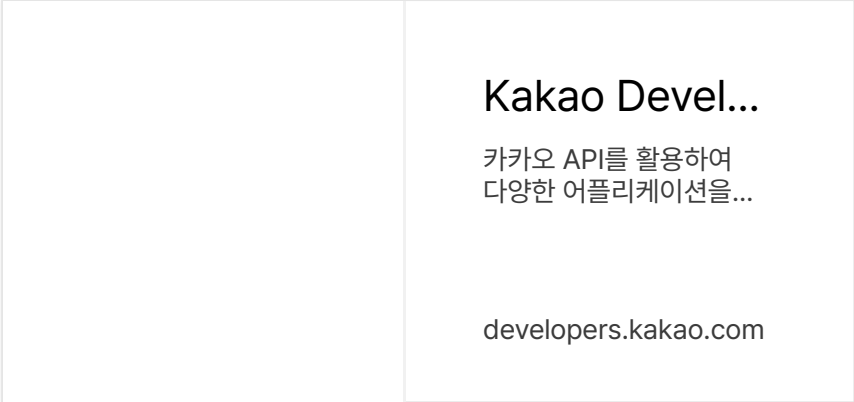
이제 기본적인 세팅은 끝났으니, 카카오 로그인 흐름을 알아보자.

3. 카카오 로그인 흐름



Kakao Login sequence with REST API

REST API를 이용한 카카오 로그인 순서는 다음과 같다.



이 순서를 따라서 카카오 로그인을 구현해보자.

4. 카카오 로그인 구현

먼저 Step1을 진행해보자.
https://kauth.kakao.com/oauth/authorize에 **GET** 요청을 보낸다. (❶)
이때, **Ajax**를 통한요청이 불가하므로, **<a>** 태그를 통한 리다이렉트로 **GET** 요청을 보낸다.

```
import { KAKAO_AUTH_URL } from
"../config/KakaoConfig";

function KakaoLogin() {
  return (
    <button type="button">
      <a href={KAKAO_AUTH_URL}>카카오 로그인</a>
    </button>
  );
}
export default KakaoLogin;
```

그러면 ❷ ~ ❺까지는 사용자와 카카오 Auth Server에서 자동으로 처리가 된다.

그 다음에 Redirect URI로 인가 코드가 전달되는데, 우리가 설정한 주소(http://localhost:3000/auth/kakao/callback)에 쿼리 파라미터 형식으로 인가 코드가 전달된다. (❻)

쿼리 파라미터로 전달된 인가코드

```
const code = new
URL(document.URL).searchParams.get("code");
```

다음과 같이 인가코드를 파싱하면 Step1이 마무리된다.

이제 Step2: 토큰 받기를 진행해보자.

POST 요청을 https://kauth.kakao.com/oauth/token에 보내면 되는데, 카카오 공식 문서에 따르면 다음과 같은 양식을 따라야 한다.

요청

헤더

이름	설명	필수
Content-type	Content-type: application/x-www-form-urlencoded;charset=utf-8 요청 데이터 타입	O

본문

이름	타입	설명	필수
grant_type	String	authorization_code로 고정	O
client_id	String	앱 REST API 키 [내 애플리케이션] > [앱 키]에서 확인 가능	O
redirect_uri	String	인가 코드가 리다이렉트된 URI	O
code	String	인가 코드 받기 요청으로 얻은 인가 코드	O
client_secret	String	토큰 발급 시, 보안을 강화하기 위해 추가 확인하는 코드 [내 애플리케이션] > [카카오 로그인] > [보안]에서 설정 가능 ON 상태인 경우 필수 설정해야 함	X

양식에 맞게 요청 헤더를 설정하고,

```
headers: {
  "Content-Type": "application/x-www-form-urlencoded;charset=utf-8",
},
```

요청 본문인 payload를 json 형식으로 다음과 같이 설정한다.

```
let payload;
if (code) {
  payload = {
    grant_type: "authorization_code",
    client_id: REST_API_KEY,
    redirect_uri: REDIRECT_URI,
    code,
    client_secret: CLIENT_SECRET,
  };
```

```
KakaoAuth.tsx:41
{grant_type: 'authorization_code', client_id: 'e01bf219117db81eedaff4840bfce65a', redirect_uri: 'http://localhost:3000/auth/kakao/callback', code: 'lkk_qa0eUYQrf0ewAbfgXa7B2WMLv2UTWXXuaTH5G_J_UUpdizmsogAAAAQKPCKbAAABkUX3Ymm37mS5Kc-sjw', client_secret: 'YjinHcCQFT51QhsY5vw6RsFZOPBJFtV4'}
  client_id: "e01bf219117db81eedaff4840bfce65a"
  client_secret: "YjinHcCQFT51QhsY5vw6RsFZOPBJFtV4"
  code: "lkk_qa0eUYQrf0ewAbfgXa7B2WMLv2UTWXXuaTH5G_J_UUpdizmsogAAAAQKPCKbAAABkUX3Ymm37mS5Kc-s
  grant_type: "authorization_code"
  redirect_uri: "http://localhost:3000/auth/kakao/callback"
▶ [[Prototype]]: Object
```

요청 본문

응답으로 다음과 같은 데이터가 오는 것을 확인할 수 있다.

```
KakaoAuth.tsx:55
▼ {data: {…}, status: 200, statusText: 'OK', headers: AxiosHeaders, config: {…}, …}
  ▶ config: {transitional: {…}, adapter: Array(3), transformRequest: Array(1), transformRespon
  ▶ data: {access_token: 'F_ySBFwoKk_w41Cv_CCm2wrU2olG3wsSAAAAAQoqJVEAAAGRRfervIh6dPOEuoNF', to
  ▶ headers: AxiosHeaders {cache-control: 'no-cache, no-store, max-age=0, must-revalidate', con
  ▶ request: XMLHttpRequest {onreadystatechange: null, readyState: 4, timeout: 0, withCredentia
    status: 200
    statusText: "OK"
  ▶ [[Prototype]]: Object
```

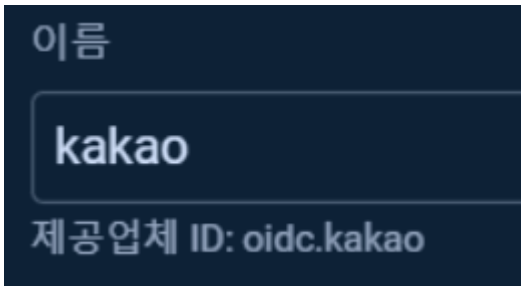
응답

이 응답이 정상적으로 오면, 토큰이 성공적으로 발행된 것이다.

이제 Step3 사용자 로그인 처리를 할 시간이다.

파이어베이스의 OpenID Connect를 통해서 로그인 처리를 할 예정인데, ID 토큰 유효성 검증, 사용자 정보 조회, 서비스 회원 정보 확인 또는 가입처리는 파이어베이스에서 자동으로 진행된다.

그러므로 파이어베이스의 코드만 짜면 되는데, OAuthProvider에 먼저 정의한 제공업체 ID를 입력한다.



oidc.kakao

이전에 구현했던 구글 로그인과는 다르게 signInWithCredential을 통해서 진행한다. (세션 유지 설정은 동일)

```
const provider = new OAuthProvider("oidc.kakao");
const credential = provider.credential({
  idToken: req.data.id_token,
});
setPersistence(authService,
browserSessionPersistence).then(() => {
  signInWithCredential(authService, credential)
    .then((result) => {
      console.log(result);

      const credential =
OAuthProvider.credentialFromResult(result);
      const acToken = credential?.accessToken;
      const idToken = credential?.idToken;
      setUsername(result.user.displayName);
    })
```

```
    .catch((error) => {
      console.log(error);
    });
});
navigate("/");
```

5. 라우팅 및 카카오 인증 컴포넌트

```
import { createBrowserRouter, RouterProvider }
from "react-router-dom";
import LoginHome from "./LoginHome";
import KakaoAuth from
"./components/Kakao.tsx/KakaoAuth";
import Profile from
"./components/Kakao.tsx/Profile";

const router = createBrowserRouter([
  {
    path: "/",
    children: [
      {
        index: true,
        element: <LoginHome />,
      },
      {
        path: "auth/kakao/callback",
        element: <KakaoAuth />,
      },
      {
        path: "profile",
        element: <Profile />,
      },
    ],
  },
]);

function App() {
  return <RouterProvider router={router} />;
}
```

```
export default App;
```

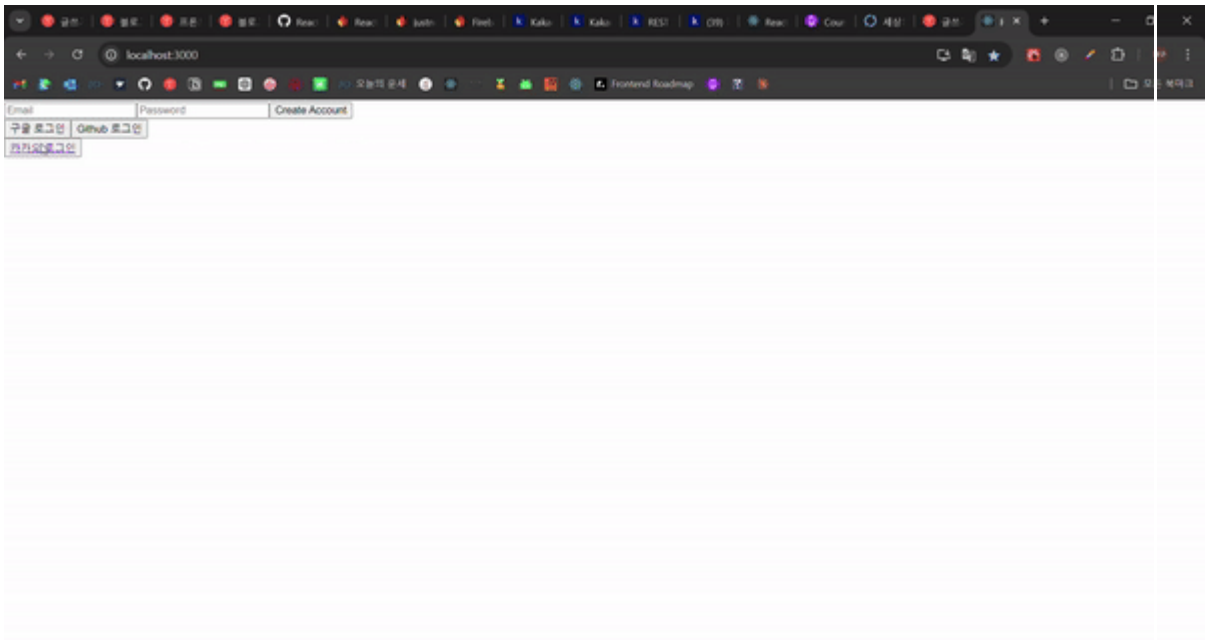
로그인 진행중... 이거말고 사실 다른 페이지를 띄우거나 아니면 로딩스피너를 띄우는 방식도 가능

```
import { useNavigate } from "react-router-dom";
import useZustandAuthStore from
"../../store/ZustandAuthStore";
import {
  CLIENT_SECRET,
  KAKAO_APP_KEY,
  REDIRECT_URI,
  REST_API_KEY,
} from "../../config/KakaoConfig";
import axios from "axios";
import { useEffect } from "react";
import {
  browserSessionPersistence,
  OAuthProvider,
  setPersistence,
  signInWithCredential,
} from "firebase/auth";
import { authService } from
"../../firebase/fbInstance";

function KakaoAuth() {
  const navigate = useNavigate();
  const setIsLoggedIn =
useZustandAuthStore((state) =>
state.setIsLoggedIn);
  const setUsername = useZustandAuthStore((state)
=> state.setUsername);
  const code = new
URL(document.URL).searchParams.get("code");

  async function getKakaoAuthToken() {
    let payload;
    if (code) {
      payload = {
```

```
        grant_type: "authorization_code",
        client_id: REST_API_KEY,
        redirect_uri: REDIRECT_URI,
        code,
        client_secret: CLIENT_SECRET,
      };
      console.log(payload);
    } else {
      alert("code doesn't exist , try again");
    }
    try {
      const req = await axios.post(
        "https://kauth.kakao.com/oauth/token",
        payload,
        {
          headers: {
            "Content-Type": "application/x-www-
form-urlencoded;charset=utf-8",
          },
        }
      );
      console.log(req);

      const provider = new
OAuthProvider("oidc.kakao");
      const credential = provider.credential({
        idToken: req.data.id_token,
      });
      setPersistence(authService,
browserSessionPersistence).then(() => {
        signInWithCredential(authService,
credential)
          .then((result) => {
            console.log(result);

            const credential =
OAuthProvider.credentialFromResult(result);
            const acToken =
credential?.accessToken;
            const idToken = credential?.idToken;
            setUsername(result.user.displayName);
          })
          .catch((error) => {
```

```
            console.log(error);
          });
      });
      navigate("/");
    } catch (err) {
      console.log(err);
    }
  }

  useEffect(() => {
    getKakaoAuthToken();
  }, []);
  return <div>로그인 진행중</div>;
}

export default KakaoAuth;
```

6. 결과



로그인이 성공적으로 완료가 되면서, 내 이름과 프로필 사진이 성공적으로 가져와졌다.

그런데, 새로고침시 프로필 사진의 깜빡임이 보인다.
이는 img를 외부 출처에서 가져오기 때문이라 이하의 문서를 통해 최적화를 진행하면 된다.

invertase/re...

A set of React Query hooks integrating wi...

react-query-firebase.invertase.dev

그런데, React와 Firebase의 authentification만을 kakao login에 사용하면 보안 이슈가 있을 수 있다.

state	String	카카오 로그인 과정 중 동일한 값을 유지하는 임의의 문자열(정해진 형식 없음) Cross-Site Request Forgery(CSRF) 공격으로부터 카카오 로그인 요청을 보호하기 위해 사용 각 사용자의 로그인 요청에 대한 state 값은 고유해야 함 인가 코드 요청, 인가 코드 응답, 토큰 발급 요청의 state 값 일치 여부로 요청 및 응답 유효성 확인 가능	X
nonce	String	OpenID Connect를 통해 ID 토큰을 함께 발급받을 경우, ID 토큰 재생 공격을 방지하기 위해 사용 ID 토큰 유효성 검증 시 대조할 임의의 문자열(정해진 형식 없음)	X

state와 nonce를 이용해서 방지하는 옵션을 추가하여야하는데, 단순히 React로는 nonce를 추가하기 어렵고, 백엔드가 있어야 하지 않을까.(NextJS도 가능)

만약 백엔드가 있다면, 보안을 위해 프론트와 백에서 각각 firebase의 access token을 검증하는 과정이 필요하다.

카카오 로그인 글 쓰다가 중간에 코로나걸려서 흐름 끊기고 글이 너무 어영부영 마무리가 되었다...
카카오 로그아웃 처리도 올려야하는데 아파서 포기

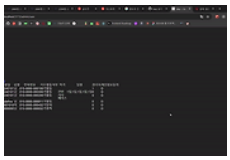
궁금한 부분이 있으면 댓글 남겨주세요.

2 구독하기

'프론트엔드 관련 > 프로젝트' 카테고리의 다른 글

[React] 비대화형 요소(td, div)에 마우스 이벤트(onFocus, onBlur) 넣기 (0)	2024.09.23
[React] figma 디자인 적용하기 (1)	2024.09.11
[React] 구글 로그인 기능 구현(with firebase 소셜 로그인 인증) (0)	2024.08.07
[React-Redux] 실제 프로젝트에 적용하고, thunk로 비동기 통신 구현 (1)	2024.05.08
React 구글맵(googleMap) 최적화(with typescript) (3)	2024.04.26

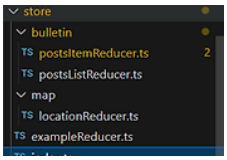
관련글



[React] 비대화...

[React] figma ...

[React] 구글 로...



[React-Redux] ...

ash9river

ash9river@gmail.com

구독하기

댓글 2

익명

비밀댓글입니다.

2024. 8. 13. 15:40

익명

비밀댓글입니다.

2024. 8. 13. 22:58

이름

비밀번호

내용을 입력하세요.

등록

github

Designed by 티스토리

© Kakao Corp.